

Vulnerability 4: Forged Review (Broken Access Control / Mass Assignment)

CWE: CWE-284 - Improper Access Control / CWE-915 - Improperly Controlled Modification of Dynamically Determined Object Attributes

OWASP: A01:2021 - Broken Access Control

CVSS: 6.5 (Medium)

Location: routes/createProductReviews.ts

Component: Backend API - Product review endpoint (PUT /rest/products/{id}/reviews)

Description

The product review endpoint inserts a new review into the MongoDB reviews collection using the author field taken **directly from the request body** (req.body.author). Although the endpoint does fetch the authenticated user via security.authenticatedUsers.from(req), it only uses that identity to detect the Juice Shop challenge. It does **not** use it to set the review's author. The stored author value therefore remains fully attacker controlled.

An attacker can intercept a review submission request and change the author field to any other user's email. The forged review is then stored and displayed as if that user had posted it. This is a combined **mass assignment** and **broken access control** vulnerability. The server fails to ensure that the client cannot assert a different identity than the one in the authenticated session.

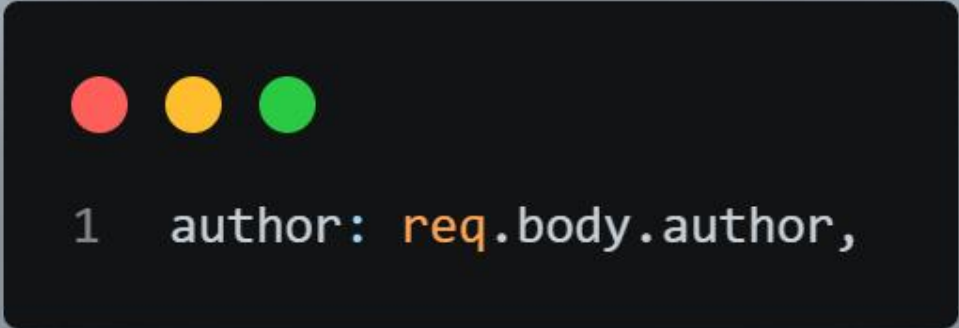
The forgedReviewChallenge code block is intentionally present to detect the attempt, but detection is not prevention. The vulnerability remains.

Vulnerable code (Backend):

```
await reviewsCollection.insert({  
  product: req.params.id,  
  message: req.body.message,  
  author: req.body.author,  
  likesCount: 0,  
  likedBy: []  
})
```



```
1  export function createProductReviews () {  
2    return async (req: Request, res: Response) => {  
3      const user = security.authenticatedUsers.from(req)  
4      challengeUtils.solveIf(  
5        challenges.forgedReviewChallenge,  
6        () => user?.data?.email !== req.body.author  
7      )  
8  
9      try {  
10       await reviewsCollection.insert({  
11         product: req.params.id,  
12         message: req.body.message,  
13         author: req.body.author,  
14         likesCount: 0,  
15         likedBy: []  
16       })  
17       return res.status(201).json({ status: 'success' })  
18     } catch (err: unknown) {  
19       return res.status(500).json(utils.getErrorMessage(err))  
20     }  
21   }  
22 }
```

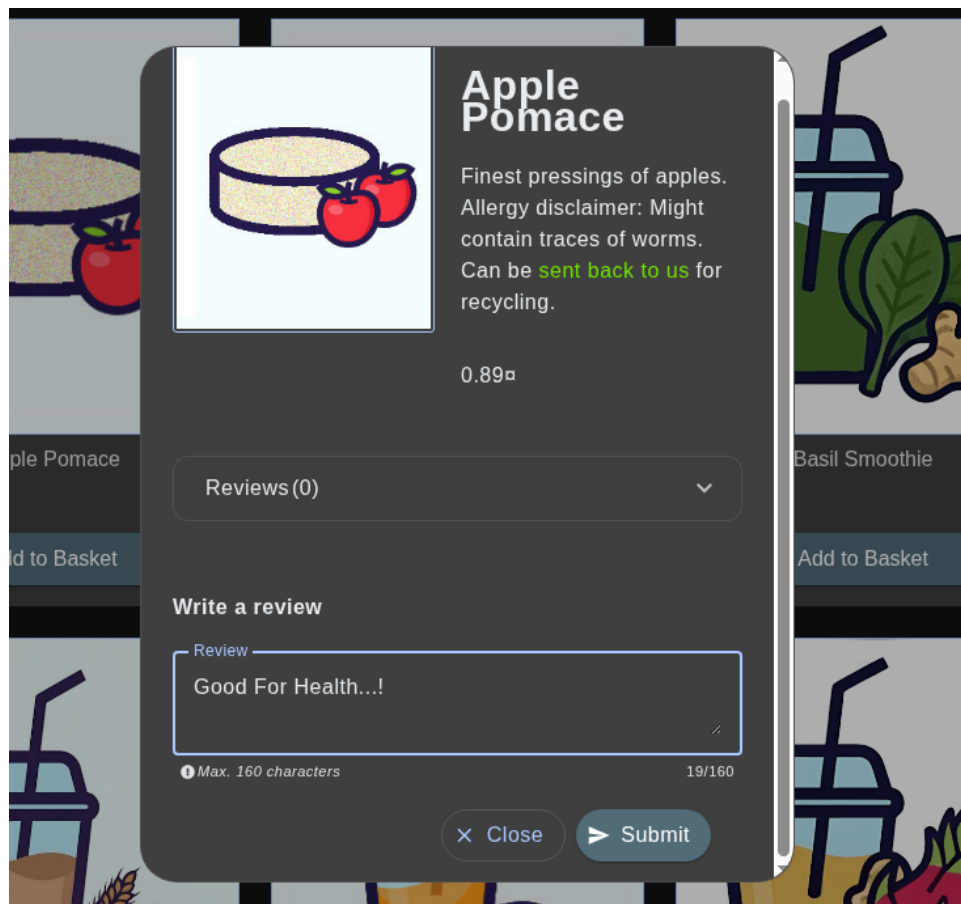


```
1  author: req.body.author,
```

The authenticated user's identity is retrieved (`security.authenticatedUsers.from(req)`) but discarded for persistence purposes. The author stored in MongoDB is whatever the client sent in the request body.

Exploit: {"message": "Forged review content", "author": "admin@juice-sh.op"}

1. Log in as a non-admin user.
2. Navigate to any product page and submit a review.
3. Intercept the PUT /rest/products/{id}/reviews request with Burp Suite or browser DevTools.
4. Observe the JSON body containing {"message": "...", "author": "jim@juice-sh.op"}.
5. Changing the author field to VICTIM.
6. Forward the modified request
7. Refresh the product page. The review now appears under the VICTIM's identity



Evidence - Before Fix

- Modify the author's field to another user's email.
- Forward the tampered request. The server responds with 201 Created and {"status": "success"}, indicating the review was stored without validating the author against the authenticated session.
- Refresh the product page. The review now appears in the product's review list attributed to another user, even though the authenticated session belongs to another user.
- Screenshot showing the intercepted request in Burp Suite with the modified author field, and the product page displaying the forged review under the new user's identity. Confirming successful impersonation through client-controlled input.

[illegible]

Response

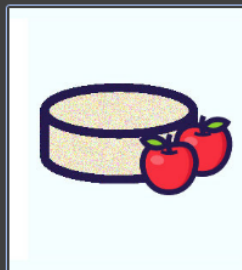
Pretty

Raw

Hex

Render

```
1 HTTP/1.1 201 Created
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 20
9 ETag: W/"14-Y53wuE/mmbSikKcT/WuaLL1N65U"
10 Vary: Accept-Encoding
11 Date: Fri, 25 Sep 2026 12:53:52 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success"
}
```



Apple Pomace

Finest pressings of apples.
Allergy disclaimer: Might contain traces of worms.
Can be [sent back to us](#) for recycling.

0.89

Reviews (2)

TESTER@gmail.com



Good For Health...!

VICTIM@gmail.com



NOT Good For Health...!, BAD BAD BAD

SAST Detection - Before Fix

Tool: Semgrep

Rule: Custom rule targeting req.body.author usage in review handlers

Run against the unmodified routes/createProductReviews.ts, Semgrep flags the interpolated query. Capture the finding count and the specific rule ID and line number flagged.

```
(semgrep-venv)-(kali㉿kali)-[~/juice-shop-lab/juice-shop]
$ semgrep scan \
--config .semgrep-rules/forged-author.yml \
--metrics=off \
routes/createProductReviews.ts \juice-shop
--json --output=semgrep-review-before.json

ooo ~/juice-shop-lab/juice-shop
Semgrep CLI

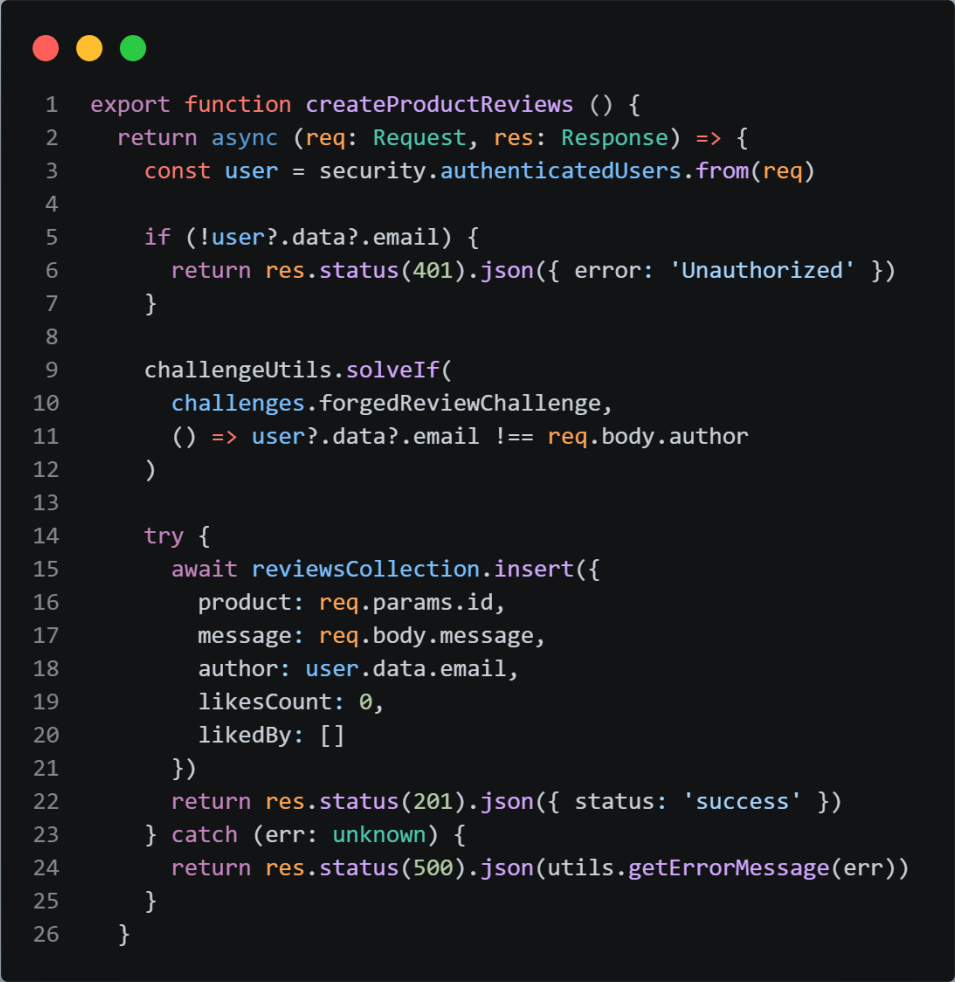
juice-shop@20.2.0 start
? Loading rules ...
Scanning 1 file (only git-tracked) with 1 Code rule:
info: Detected Node.js version v24.19.0 (~$24.19.0)
1 CODE RULES as linux (~$24.19.0)
1 Scanning 1 file. x64 (~$24.19.0)
info: Configuration default validated (~$24.19.0)
1 SUPPLY CHAIN RULES 1 of 21 are initialized (~$24.19.0)
info: All dependencies in ./package.json are satisfied (~$24.19.0)
1 No rules to run. server.js is present (~$24.19.0)
info: Required file index.html is present (~$24.19.0)
info: Required file styles.css is present (~$24.19.0)
1 PROGRESS red file main.js is present (~$24.19.0)
info: Required file polyfills.js is present (~$24.19.0)
1 100% 0:00:00 $/ is
info: Port 3000 is available (~$24.19.0)
info: Domain https://www.alchemy.com/ is reachable (~$24.19.0)
warn: Environment variable ALCHEMY_API_KEY is not present (warn:
Honey Pot" challenge will not work as intended w
https://howto-web3.owasp-juice.shop for instructio
✓ Scan completed successfully.1434/v1 is not reachable (warn:
• Findings: 1 (1 blocking) "challenge will not work as in
• Rules run: 1 "challenge will not work as
• Targets scanned: 1 "challenge will not work as intended witho
• Parsed lines: ~100.0% "challenge will not work as in
• No ignore information availablep-juice.shop for instruction
Ran 1 rule on 1 file: 1 finding.ooo
```

Secure Coding Fix

The server must derive the review's author from the **authenticated session**, not from client-supplied input. The user object already obtained at the top of the function is the correct source of truth.

```
if (!user?.data?.email) {  
    return res.status(401).json({ error: 'Unauthorized' })  
}
```

author: user.data.email,



```
1  export function createProductReviews () {  
2    return async (req: Request, res: Response) => {  
3      const user = security.authenticatedUsers.from(req)  
4  
5      if (!user?.data?.email) {  
6        return res.status(401).json({ error: 'Unauthorized' })  
7      }  
8  
9      challengeUtils.solveIf(  
10        challenges.forgedReviewChallenge,  
11        () => user?.data?.email !== req.body.author  
12      )  
13  
14      try {  
15        await reviewsCollection.insert({  
16          product: req.params.id,  
17          message: req.body.message,  
18          author: user.data.email,  
19          likesCount: 0,  
20          likedBy: []  
21        })  
22        return res.status(201).json({ status: 'success' })  
23      } catch (err: unknown) {  
24        return res.status(500).json(utils.getErrorMessage(err))  
25      }  
26    }  
}
```



```
1 if (!user?.data?.email) {  
2   return res.status(401).json({ error: 'Unauthorized' })  
3 }
```



```
1   author: user.data.email,
```

Why this works: The author's identity is now bound to the server-verified session. Any author's value in the request body is ignored entirely. An attacker can still send, but the review will be stored under the authenticated user's actual email.

Evidence - After Fix

```
6ILRFU1RFUkBnbwFpbC5jb20iLCJwYXNzd29yZCI6IjcoMwQ4NmM3ODg4YjNlMzcwN2I1MjQyNzBiYzBkZjc1Iiw  
icm9sZSI6ImNlc3RvbWVyIiwZGVsdXhlVG9rZW4iOiIiLCJzYXN0TG9naW5JcCI6IjAuMC4wLjAiLCJwcm9maWw  
lSw1hZ2UiOiIvYXNzZXRzL3B1YmxyYy9pbwFnZXMvdXBsb2Fkcy9kZWZhdw0LnN2ZyIsInRvdHBTZWNyZXQiOiI  
iLCJpc0FjdGZSI6dHJlZSwiY3JlYXRlZEF0IjoIMjAyNiOwOSoyNSAxMjooO0ToOMi4zMdMgKzAwOjAwIiwidXB  
kYXRlZEF0IjoIMjAyNiOwOSoyNSAxMjooO0ToOMi4zMdMgKzAwOjAwIiwZGVsZXRlZEF0IjpudWxsSfSwiYmIkJjo  
2LCJpYXQiOjE3OTAzNDA1ODR9.sFCGq0XBJ9z00pRQYk0pmApsJJKL-HOU9BxzBHMx8213s2VoZ7OWicMQD_c7MC  
VjEYd2dQpM2lgIh6t05CkfgLJnfTVkyUuGL25fXXXGUXTP0o8ENEQg4S76DZUxwTYT2aln9-1l8tj9nDCrI5Hn  
vYpTrMHdlrqgh3IXD2Btk; cookieconsent_status=dismiss; continueCode=  
KQabVVENkByjq902xgyowrXb45wGn0SB0aL8m1pzYLPQKJmZ6D37neRqyn3x  
19 Connection: keep-alive  
20  
21 {  
  "message": "NOT Good For Health...!, BAD BAD BAD",  
  "author": "VICTIM@gmail.com"  
}
```

Response

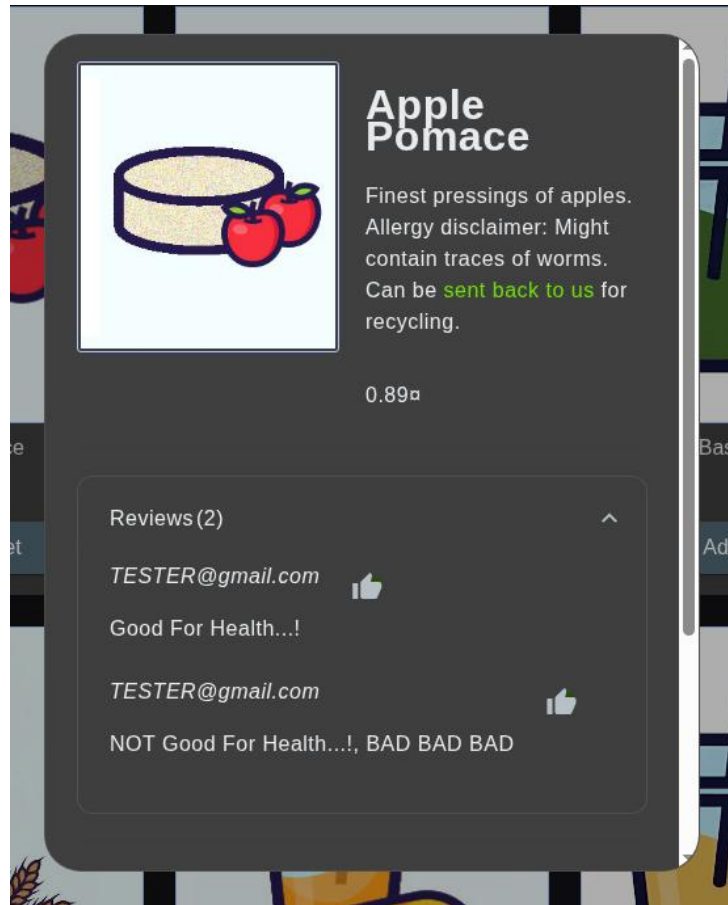
Pretty

Raw

Hex

Render

```
1 HTTP/1.1 201 Created
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 20
9 ETag: W/"14-Y53wuE/mmbSikKcT/WualL1N65U"
10 Vary: Accept-Encoding
11 Date: Fri, 25 Sep 2026 14:18:18 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success"
}
```



The server returns 401 Unauthorized or ignores the forged author field and stores the review under the authenticated user's actual identity. The forged review does not appear under forged name.

SAST Detection - After Fix

Re-running the same scan on the patched routes/createProductReviews.ts shows the SQL injection finding is gone.

```
(semgrep-venv)-(kali㉿kali)-[~/juice-shop-lab/juice-shop]
$ semgrep scan \
  --config .semgrep-rules/forged-author.yml \
  --metrics=off \
  routes/createProductReviews.ts \
  --json --output=semgrep-review-after.json
```

Semgrep CLI

Scanning 1 file (only git-tracked) with 1 Code rule:

CODE RULES
Scanning 1 file.

SUPPLY CHAIN RULES
No rules to run.

PROGRESS

100% 0:00:00

Scan Summary

✓ Scan completed successfully.

- Findings: 0 (0 blocking)
- Rules run: 1
- Targets scanned: 1
- Parsed lines: ~100.0%
- No ignore information available

Ran 1 rule on 1 file: 0 findings.

✦ If Semgrep missed a finding, please send us feedback to let us know!
See <https://semgrep.dev/docs/reporting-false-negatives/>

Before / After Summary

Metric	Before Fix	After Fix
Semgrep finding count on createProductReviews.ts	1	0
Forged review as admin@juice-sh.op	Stored successfully	Ignored; stored under real user
Author source	req.body.author (client)	user.data.email (JWT session)
Unauthenticated requests	Accepted (no guard)	Rejected with 401
Root cause present	Yes	No
CWE-284 / CWE-915 compliant	No	Yes